

Week 04 Explained Student Worksheet

Term-Document Incidence Matrix

Name _____ Student ID _____ Group _____

Represent the five-document collection using one binary row per term. This matrix becomes the input to Boolean retrieval in Week 5.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Assign consistent document IDs and a fixed column order.
- Construct a collection vocabulary from preprocessed text.
- Implement `build_matrix(documents)` using 0/1 occurrence values.
- Verify selected matrix rows manually.

Understand the concepts

A term-document incidence matrix records whether a term occurs in each document. Rows correspond to terms; columns correspond to document IDs. A value of 1 means at least one occurrence, and 0 means absence. Repetition within one document does not change a binary entry.

The program stores the matrix as a dictionary from a term to a list of bits. The separate `doc_names` list specifies which document each position represents. Keep that list with the matrix: a vector without its column order is ambiguous.

Following 5.py, the program tokenizes each document, collects unique terms, and checks membership for every term/document pair. Sorted terms make printing repeatable; sorted document IDs make the vector interpretation stable. This straightforward representation is useful for learning, but a large sparse collection would store many zeros.

Predict before running

Using document order `d1`, `d2`, predict the python row for `d1="python data"` and `d2="java data"`.

Prepare your terminal

Extract the course and open a terminal inside `Week_04_Term_Document_Matrix`. Keep docs and data beside the Python files. The main demonstration is `lab04_solution.py`; your exercise starter is `lab04_exercises.py`. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab04_solution.py
py -3.13 lab04_exercises.py
```

Use `py -3` or `python` instead if that is your working Python 3.10+ command.

Worked example with explanation

Build a tiny incidence matrix by hand before running the complete five-document program.

```
documents = {"d1": ["python", "data", "data"],
             "d2": ["java", "data"]}
for term in ["data", "java", "python"]:
    row = [1 if term in documents[name] else 0
           for name in sorted(documents)]
    print(term, row)
```

Expected output from an executed example

```
data [1, 1]
java [0, 1]
python [1, 0]
```

How each stage produces the result

1. The example already provides token lists to isolate matrix construction. In the full lab, raw document strings are first passed through `tokenize`.
2. The outer loop visits one term at a time. Each term owns a row, while positions in that row refer to the fixed sorted document order.
3. A membership test produces 1 for presence and 0 for absence. Two data occurrences in `d1` still produce one presence bit.
4. Keep labels with vectors. Without document-order labels, `[1, 0]` says that a term appears somewhere, but does not identify the matching document.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab04_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def build_matrix(documents):
    doc_names = sorted(documents)
    words_in_docs = {}
    all_words = []
    for name in doc_names:
        words_in_docs[name] = tokenize(documents[name])
        all_words.extend(words_in_docs[name])
    matrix = {}
    for term in sorted(set(all_words)):
        row = []
        for name in doc_names:
            row.append(1 if term in words_in_docs[name] else 0)
        matrix[term] = row
    return matrix, doc_names
```

Code walkthrough

1. doc_names is the column contract. Every row is built by looping over that same list, and the function returns it alongside the matrix.
2. words_in_docs caches each token list. all_words accumulates occurrences from the whole collection; sorted(set(all_words)) selects unique row labels.
3. For each vocabulary term, row starts empty and receives exactly one bit per document. matrix maps a term to its complete row.
4. This representation answers presence questions but does not retain frequency or positions. For a large collection, many zero entries can make a full matrix expensive.

If python is repeated three more times in doc1, which incidence bits should change? Explain why.

Connection to the course

Week 5 combines these binary rows to answer Boolean queries. Stable document order is essential because coordinate-wise operations assume the same document at each position.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Keep the supplied five documents unchanged for the first run.

Use the original five documents so the baseline rows are comparable with the previous lab.

Step 2 Tokenize each document using the Week 3 function.

Check token lists before inspecting the matrix; a missing token upstream produces a missing matrix entry.

Step 3 Build `all_words`, then convert it to a set and sort it to obtain the vocabulary.

Count occurrences and unique terms separately. Only unique terms become row keys.

Step 4 For each term, append one binary value for each ID in `sorted(documents)`.

Use one document order for every row. A row must have exactly as many entries as there are document IDs.

Step 5 Return the matrix and document order. Print labeled columns.

Return the order as data rather than rebuilding labels independently when printing.

Step 6 Compare python, java, machine, and data rows with manual predictions, then complete the starter.

Predict four selected rows on paper, then compare each bit with the corresponding document text.

Test record

Check to demonstrate	Actual observation	Pass or revise
Every row length equals the document count		
Selected rows agree with manual presence checks		
Repeated occurrences do not turn a binary value into a count		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Basic: predict the rows for learning and analysis before running the program.
- Implementation: complete `build_matrix` in the starter, including vocabulary construction.
- Debugging: deliberately reverse the printed column labels without reversing the vectors. Explain the incorrect interpretation this produces.
- Interpretation: add three more occurrences of `python` to `doc1`. State which matrix entries change and why.

Diagnose common errors

The printed IDs identify the wrong matches. Column labels were reordered without reordering vectors. Preserve the returned `doc_names` list.

Some cells contain 2 or 3. The incidence matrix represents presence, not TF. Use a membership test to choose exactly 0 or 1.

Submit these items

- Your completed matrix builder and a labeled matrix for the original five documents.
- Manual predictions and checks for four selected terms.
- A short comparison of a token count and a binary occurrence value.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

What does one row represent?

Why must document order remain fixed?

Optional challenge

Count the number of 1s and 0s and calculate the proportion of nonzero entries. Explain why this density measure matters when choosing a representation.

Source for this companion

Based on the supplied Week 04 lab sheet, shipped program, exercise starter, and sample data. The complete main program and `expected_output.txt` in this folder are the references for checking your baseline run.